
EVOLUTIONARY COMPUTING: STANDARD ASSIGNMENT - TASK II

21.09.2026

Read this whole document before starting your assignment!

1 Introduction

In this assignment you will develop, apply, and compare Evolutionary Algorithms for the task of neuroevolution using the `ariel` framework, evolving a controller for a fixed robot body from the 'John Set', gaining hands-on experience with Evolutionary Computation along the way. The assignment is to be done in groups of 4. **The deadline -not negotiable- is as follows:**

13.10.2026, 9:00

Each day of delay in the submission will result in a deduction of 0.5 points from your Task grade.

2 Resources

This is the same `ariel` framework and repository you used for Assignment 1 - no new installation is required. All the resources (files) you need are available on GitHub, including code and documentation. Starter code for this task is available at `A2_template_2026.py`. It spawns a robot, drives it with a neural network with RANDOM weights, and reports how close the robot ended up to a target - there is deliberately no evolution in it. Building the EA is the assignment; see the "YOUR JOB" section at the bottom of that file.

3 Preparation

- You should already have `ariel` installed from Assignment 1. Pull the repository to get `A2_template_2026.py`, then confirm by running it.
- Running it also shows you how the controller contract, the fitness function, and the world/body are set up.

4 Supplementary material

- `ariel` documentation - <https://ci-group.github.io/ariel/>
- 'John Set' body library - `ariel.body_phenotypes.robogen_lite.prebuilt_robots.john_set`
- Ready-made fitness variants for targeted locomotion, if useful for your research question - `ariel.simulation.tasks.targeted_locomotion`.

5 Task

- **WHAT:** Implement an EA, built on `ariel.ec`, that evolves the weights of a neural-network controller so a fixed body from the 'John Set' walks from its spawn position to a fixed target position. Investigate one aspect of your EA (e.g. mutation, crossover, parent selection, survivor selection, population size, or genotype representation) as your research question.
- **Body:** free choice from the 'John Set' (see Supplementary material above). Fixed for your whole assignment.
- **World:** free choice from `ariel.simulation.environments`, fixed for your whole assignment, with one exception: `SimpleTiltedWorld` is not supported for this task - bodies from the 'John Set' slide off the platform under a neutral gait before reaching the far side. `SimpleFlatWorld`, `RuggedTerrainWorld`, `CraterTerrainWorld`, `AmphitheatreTerrainWorld`, and `OlympicArena` are all valid choices.
- **Controller:** a neural network whose weights you evolve, see `A2_template_2026.py`. CPG-based controllers are not supported for this task.
- **Fitness:** the Euclidean distance, in the flat ground plane, between the core body's final position and a fixed target position (lower is better). Think carefully about what information your controller needs to solve this - a controller with no signal telling it where the target is, or nothing to drive rhythmic movement with, has very little to work with.

`ariel.simulation.tasks.targeted_locomotion` has several ready-made variants if you want to build on one for your research question: plain remaining distance, distance reduced from the starting position (so a robot that starts closer is not rewarded for standing still), distance combined with a control-effort penalty, a version that penalises falling over, one that penalises a wandering path relative to a straight line, and one that rewards reaching the target quickly rather than just closely. You are free to argue for a different metric in your report, as long as you define it clearly.

- **HOW:** Design an experiment to answer your research question. Repeat your final experiments at least 5 times independently and present statistics (mean and spread) based on these runs. Compare against a baseline (e.g. random search, or a fixed non-evolved controller) at the same evaluation budget.
- **WHAT TO HAND IN:** The report as specified below and your code.
- **HOW TO HAND IN:** Submit a single compressed file named `groupnumber.zip`, containing a folder with the same name, including the report as `groupnumber.pdf` and your code.

6 Rules and guidelines

- You are NOT allowed to make changes to the source code of the `ariel` framework (anything in `src/ariel`). Making such changes would be qualified as fraud, leading you to FAIL the assignment.
- You may use `ariel`'s built-in EC utilities (e.g. `ariel.ec`) as building blocks, but the evolutionary search itself must be your own design and implementation, not an off-the-shelf black-box optimizer (e.g. `nevergrad`, CMA-ES libraries).
- Compare your algorithm(s) with a line-plot across generations, showing the average/std of fitness over your independent runs.

7 Report structure

Maximum 6 pages (excluding cover page and bibliography), GECCO19 template.

Cover page: name of the course; number and name of the Task; team number/name; team members' names and student ID numbers; date.

- **Introduction** (a clear research question or goal, cf. Reporting Research document).
- **Methods:** explain how your algorithm works and its motivation, parameter settings, experimental setup, fitness function, budget. Make sure everything is reproducible.
- **Results and discussion:** discuss your results and what they show about your research question.
- **Conclusions**
- **Literature list / bibliography**

8 Tips

- Test your algorithm first using a small population and short simulation durations.
- This is a complex search problem - do not expect convergence in a handful of generations. Run until your fitness curve plateaus, and treat that plateau, not a fixed generation count, as your stopping criterion.
- Physics-based fitness evaluation is much slower than pure body-scoring - budget your time accordingly, and test your pipeline at a small scale before committing to a full run.
- Be organized concerning the output files of your experiments.
- Parametrize your seed (e.g. a command-line argument or environment variable) so running your final ≥ 5 -seed repeats is a single script invocation per configuration, not a manual edit-and-rerun each time.

9 Grading

Your grade is based on the provided report. We will check the code for correctness and coherence with the report, but pure performance is not part of the grade. Treat the code mainly as a source of data for the report.

Each Task is worth a maximum of 10 points, and the final assignment grade is calculated as: 50% (Assignment 1) + 50% (Assignment 2 - you're reading it now).